

Guide Pratique

Ateliers MLOps : Projet Titanic

Ateliers 2 · 3 · 4 · 5 · 6 · 7

Realisé par :
Souissi Oumaima
&
Maaoui Ghofran

Atelier 2 — Modularisation du Code

Prérequis : Script Jupyter fonctionnel.

Objectif

Transformer le notebook Jupyter en code Python modulaire et réutilisable.

Étapes

1. Créer src/model_pipeline.py

Implémenter les 6 fonctions suivantes :

<code>prepare_data()</code>	Charge le CSV, normalise colonnes, gère NaN, encode, split train/test
<code>train_model()</code>	Entraîne LogisticRegression + logue dans MLflow
<code>evaluate_model()</code>	Calcule accuracy + classification_report
<code>save_model()</code>	Sauvegarde le modèle avec joblib
<code>load_model()</code>	Charge un modèle depuis le disque
<code>retrain_model()</code>	Réentraîne avec hyperparamètres configurables (C, solver...)

2. Créer main.py

Fichier d'entrée CLI avec argparse :

```
python main.py --prepare # Prépare les données
python main.py --train   # Entraîne + sauvegarde
python main.py --evaluate # Évalue le modèle
python main.py --all      # Pipeline complet (défaut)
```

3. Correctif CSV — Colonnes en minuscules

Le fichier titanic.csv utilise des noms en minuscules (survived, sex...). Ajouter dans `prepare_data()` :

```
df.columns = df.columns.str.strip().str.capitalize()
cols_to_drop = ['Unnamed: 0', 'PassengerId', 'Name', 'Ticket', 'Cabin']
```

Livrables

- ✓ src/model_pipeline.py — fonctions modularisées avec docstrings
- ✓ main.py — CLI avec --prepare, --train, --evaluate, --all

Atelier 3 — Création du Makefile

Prérequis : Code modulaire fonctionnel (Atelier 2).

Objectif

Automatiser toutes les tâches du projet (ML + CI/CD) via des commandes make simples.

Cibles du Makefile

Cible	Commande exécutée	Description
install	<code>pip install -r requirements.txt</code>	Installe toutes les dépendances
prepare	<code>python main.py --prepare</code>	Prépare les données
train	<code>python main.py --train</code>	Entraîne le modèle
evaluate	<code>python main.py --evaluate</code>	Évalue les performances
all	<code>python main.py --all</code>	Pipeline ML complet
format	<code>black src/ main.py app.py</code>	Formate le code (CI)
lint	<code>flake8 src/ main.py app.py</code>	Qualité du code (CI)
security	<code>bandit -r src/</code>	Analyse de sécurité (CI)
run-api	<code>uvicorn app:app --reload --port 8000</code>	Démarre l'API FastAPI
mlflow-ui	<code>mlflow ui --backend-store-uri sqlite:///mlflow.db --port 5000</code>	Interface MLflow

Note importante

⚠ **CI/CD automatique** : Le jour de la validation, toute modification dans le projet doit déclencher automatiquement toutes les étapes du Makefile. Cette tâche est notée dans la grille finale.

Livrables

- ✓ Makefile — install, pipeline ML, CI (format/lint/security), API, MLflow

Atelier 4 — Exposition via FastAPI

Prérequis : Modèle entraîné et sauvegardé + Makefile fonctionnel.

Objectif

Exposer predict() comme service REST. Excellence : exposer aussi retrain().

Étapes

1. Installer les dépendances

```
pip install fastapi uvicorn
```

Ajouter fastapi et uvicorn dans requirements.txt.

2. Créer app.py

Le fichier app.py doit contenir :

- Chargement du modèle depuis models/titanic_model.pkl
- Route POST /predict — reçoit les données passager, retourne la prédiction
- Route POST /retrain (excellence) — réentraîne avec nouveaux hyperparamètres

3. Exemple de requête /predict

Corps JSON envoyé :

```
{ "Pclass": 1, "Age": 29.0, "SibSp": 0, "Parch": 0,
  "Fare": 211.3, "Sex_male": false,
  "Embarked_Q": false, "Embarked_S": true }
```

Réponse retournée :

```
{ "prediction": 1, "survived": "Yes" }
```

4. Lancer le serveur

```
uvicorn app:app --reload --host 0.0.0.0 --port 8000
```

Documentation Swagger : <http://127.0.0.1:8000/docs>

5. Ajouter dans le Makefile

```
run-api:
uvicorn app:app --reload --host 0.0.0.0 --port 8000
```

Livrables

- ✓ app.py — /predict et /retrain (excellence)
- ✓ Documentation Swagger — <http://127.0.0.1:8000/docs>
- ✓ Commande make run-api dans le Makefile

Atelier 5 — Introduction à MLflow

Prérequis : Modèle opérationnel et API REST (Ateliers 2, 3, 4).

Objectif

Intégrer MLflow pour suivre les expériences (params, métriques, modèles) et visualiser via l'interface web.

Étapes

1. Installer MLflow

```
pip install --upgrade mlflow
```

En cas d'erreur protobuf (ImportError: google.protobuf.service) → upgrade mlflow résout le problème.

Ajouter mlflow dans requirements.txt.

2. Lancer l'interface MLflow

```
mlflow ui --backend-store-uri sqlite:///mlflow.db --host 0.0.0.0 --port 5000
```

Accès : <http://localhost:5000> — Backend SQLite recommandé (piste d'excellence).

3. Intégrer MLflow dans model_pipeline.py

Appel MLflow	Fonction	Rôle
<code>mlflow.set_experiment()</code>	<code>train_model()</code>	Crée l'expérience "Titanic_Survival"
<code>mlflow.start_run()</code>	<code>train + retrain</code>	Ouvre un run nommé
<code>mlflow.log_param()</code>	<code>train + retrain</code>	Logue model_type, max_iter, C, solver...
<code>mlflow.log_metric()</code>	<code>evaluate_model()</code>	Logue accuracy du modèle
<code>mlflow.sklearn.log_model()</code>	<code>train + retrain</code>	Sauvegarde le modèle comme artefact

4. Exemple de code — train_model()

```
mlflow.set_experiment('Titanic_Survival')
with mlflow.start_run(run_name='LogisticRegression_Training'):
    mlflow.log_param('model_type', 'LogisticRegression')
    mlflow.log_param('max_iter', 1000)
    mlflow.log_param('random_state', 42)
    model = LogisticRegression(max_iter=1000, random_state=42)
    model.fit(X_train, y_train)
    mlflow.sklearn.log_model(model, 'model')
```

5. Exemple de code — evaluate_model()

```
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
with mlflow.start_run(run_name='Evaluation', nested=True):
    mlflow.log_metric('accuracy', accuracy)
```

6. Ajouter mlflow-ui dans le Makefile

```
mlflow-ui:  
mlflow ui --backend-store-uri sqlite:///mlflow.db --host 0.0.0.0 --port 5000
```

Pistes d'excellence

- ✓ Backend SQLite : --backend-store-uri sqlite:///mlflow.db
- ✓ retrain_model() avec C, solver, max_iter configurables
- ✓ Endpoint /retrain exposé via FastAPI

Livrables

- ✓ src/model_pipeline.py — MLflow complet (params + metrics + model)
- ✓ Commande make mlflow-ui avec backend SQLite
- ✓ Interface MLflow sur http://localhost:5000

Atelier 6 — Conteneurisation avec Docker

Prérequis : Application FastAPI fonctionnelle et MLflow configuré (Ateliers 4 & 5).

Objectifs

- Créer une image Docker pour conteneuriser l'application FastAPI
- Intégrer les artefacts MLflow dans le conteneur
- Publier l'image sur Docker Hub

Étapes

Étape 1 — Installer Docker

Installer Docker Desktop (Windows) ou Docker Engine (Linux/Ubuntu) depuis le site officiel.

Vérifier l'installation :

```
docker --version
```

Étape 2 — Créer le Dockerfile

Créer un fichier Dockerfile à la racine du projet avec le contenu suivant :

```
FROM python:3.11-slim

WORKDIR /app

# Copier les fichiers du projet
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

# Exposer le port FastAPI
EXPOSE 8000

# Lancer l'application FastAPI
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Étape 3 — Construire l'image Docker

Construire l'image avec un nom personnalisé (format obligatoire : prenom_nom_classe_mlops) :

```
docker build -t Oumaima_Souissi_DS2_mlops .
```

Vérifier que l'image est bien créée :

```
docker images
```

Étape 4 — Tester l'image localement

Lancer le conteneur et mapper le port 8000 :

```
docker run -p 8000:8000 Oumaima_Souissi_DS2_mlops
```

Tester l'API dans le navigateur ou via curl :

```
# Documentation Swagger
http://localhost:8000/docs

# Test rapide via curl
curl -X POST "http://localhost:8000/predict" \
  -H "Content-Type: application/json" \
  -d '{"Pclass":1,"Age":29,"SibSp":0,"Parch":0,"Fare":211.3,
    "Sex_male":false,"Embarked_Q":false,"Embarked_S":true}'
```

Étape 5 — Pousser l'image sur Docker Hub

1. Se connecter à Docker Hub :

```
docker login
```

2. Taguer l'image avec votre nom d'utilisateur Docker Hub :

```
docker tag Oumaima_Souissi_DS2_mlops dockerhub_oumaima/Oumaima_Souissi_DS2_mlops:latest
```

3. Pousser l'image :

```
docker push dockerhub_oumaima/Oumaima_Souissi_DS2_mlops:latest
```

4. Vérifier sur <https://hub.docker.com> que l'image est bien publiée.

Étape 6 — Automatisation avec le Makefile

Ajouter ces 3 cibles dans le Makefile :

Cible	Commande	Description
docker-build	docker build -t Oumaima_Souissi_DS2_mlops .	Construit l'image Docker
docker-run	docker run -p 8000:8000 Oumaima_Souissi_DS2_mlops	Lance le conteneur
docker-push	docker push dockerhub_oumaima/Oumaima_Souissi_DS2_mlops:latest	Publié sur Docker Hub

Contenu du Makefile — section Docker

```
IMAGE_NAME = Oumaima_Souissi_DS2_mlops
DOCKER_USER = dockerhub_oumaima

docker-build:
docker build -t $(IMAGE_NAME) .

docker-run:
docker run -p 8000:8000 $(IMAGE_NAME)

docker-push:
docker tag $(IMAGE_NAME) $(DOCKER_USER)/$(IMAGE_NAME):latest
docker push $(DOCKER_USER)/$(IMAGE_NAME):latest
```


Livrables

- ✓ Dockerfile — fonctionnel, copie projet + installe deps + lance uvicorn
- ✓ Image Docker disponible localement (docker images)
- ✓ Image publiée sur Docker Hub
- ✓ Makefile — cibles docker-build, docker-run, docker-push

Atelier 7 — Monitoring avec MLflow, Elasticsearch & Kibana

Prérequis : WSL + Ubuntu, Docker installé, Application FastAPI + MLflow fonctionnels.

Objectifs

- Monitorer le modèle ML en enregistrant métriques et artefacts avec MLflow
- Envoyer les logs vers Elasticsearch pour les stocker
- Visualiser les données en temps réel avec Kibana

Architecture de la stack monitoring

Outil	Port	Rôle
MLflow	localhost:5000	Suivi des expériences, params et métriques
Elasticsearch	localhost:9200	Stockage des logs et métriques
Kibana	localhost:5601	Visualisation et tableaux de bord

Étapes

Étape 1 — Installer Docker Compose (dans WSL/Ubuntu)

```
sudo apt install -y docker.io docker-compose
docker-compose --version # vérifier l'installation
```

Étape 2 — Créer docker-compose.yml

Créer ce fichier à la racine du projet :

```
version: '3.8'

services:
  elasticsearch:
    image: elasticsearch:8.12.0
    container_name: elasticsearch
    environment:
      - discovery.type=single-node
      - xpack.security.enabled=false
      - ES_JAVA_OPTS=-Xms512m -Xmx512m
    ports:
      - "9200:9200"
    networks:
      - monitoring

  kibana:
    image: kibana:8.12.0
    container_name: kibana
    ports:
      - "5601:5601"
    environment:
      - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
    depends_on:
      - elasticsearch
```

```
networks:
  - monitoring

networks:
  monitoring:
    driver: bridge
```

Étape 3 — Lancer la stack

```
docker-compose up -d
docker ps                # vérifier que les 2 conteneurs tournent
curl http://localhost:9200 # tester Elasticsearch
```

Étape 4 — Installer la bibliothèque Python

```
pip install elasticsearch psutil
```

Ajouter dans requirements.txt : elasticsearch et psutil

Étape 5 — Adapter model_pipeline.py

Ajouter la connexion Elasticsearch et l'envoi des logs après chaque run MLflow :

```
from elasticsearch import Elasticsearch
from datetime import datetime

def get_es_client():
    return Elasticsearch('http://localhost:9200')

def log_to_elasticsearch(run_id, params, metrics):
    es = get_es_client()
    doc = {
        'timestamp': datetime.utcnow().isoformat(),
        'run_id': run_id,
        'params': params,
        'metrics': metrics
    }
    es.index(index='mlflow-metrics', document=doc)
    print('Logs envoyés vers Elasticsearch.')
```

Puis appeler log_to_elasticsearch() à la fin de train_model() :

```
with mlflow.start_run(run_name='LogisticRegression_Training') as run:
    # ... entraînement + log params + log metrics ...
    # Envoi vers Elasticsearch
    log_to_elasticsearch(run.info.run_id, params, metrics)
```

Étape 6 — Démarrer MLflow en mode serveur

```
mlflow server \
  --backend-store-uri sqlite:///mlflow.db \
  --default-artifact-root ./mlruns \
  --host 0.0.0.0 --port 5000
```

Dans un autre terminal, lancer le pipeline :

```
python main.py --all
```

Étape 7 — Ajouter au Makefile

```
monitoring-up:
docker-compose up -d

monitoring-down:
docker-compose down

mlflow-server:
mlflow server --backend-store-uri sqlite:///mlflow.db \
--default-artifact-root ./mlruns --host 0.0.0.0 --port 5000
```

Étape 8 — Visualiser dans Kibana

1. Ouvrir <http://localhost:5601>
2. Aller dans Stack Management → Index Patterns
3. Cliquer Create index pattern → taper mlflow-metrics → valider
4. Aller dans Discover pour explorer les logs envoyés

Excellence — Monitorer les ressources système

Ajouter le monitoring CPU, RAM et disque avec psutil :

```
import psutil

def log_system_metrics():
    es = get_es_client()
    doc = {
        'timestamp': datetime.utcnow().isoformat(),
        'type': 'system_metrics',
        'cpu_percent': psutil.cpu_percent(interval=1),
        'ram_percent': psutil.virtual_memory().percent,
        'ram_used_gb': round(psutil.virtual_memory().used / 1e9, 2),
        'disk_percent': psutil.disk_usage('/').percent
    }
    es.index(index='system-metrics', document=doc)
```

Créer ensuite un index pattern system-metrics dans Kibana pour visualiser les métriques système.

Visualisation — Dashboard MLOps Titanic

Après avoir configuré les Data Views (mlflow-metrics et system-metrics) dans Kibana, deux visualisations ont été créées et regroupées dans un dashboard nommé 'Dashboard MLOps Titanic'.

Data Views créées dans Kibana

Data View	Champ timestamp	Données
mlflow-metrics	timestamp	Runs MLflow, params, accuracy
system-metrics	timestamp	CPU %, RAM %, disque %

Visualisation 1 — Accuracy par modèle (Bar Chart)

Ce graphique compare l'accuracy des 3 modèles entraînés (LogisticRegression, RandomForest, SVC) sur l'axe du temps. Chaque barre représente un run MLflow.

Index	mlflow-metrics
Type	Bar vertical
Axe X	timestamp (date histogram par 30s)
Axe Y	Median of metrics.accuracy
Breakdown	params.model_type.keyword
Légende	● LogisticRegression ● RandomForest ● SVC

Résultats observés dans le dashboard :

- LogisticRegression — accuracy ≈ 0.78
- RandomForest — accuracy ≈ 0.78
- SVC — accuracy ≈ 0.61

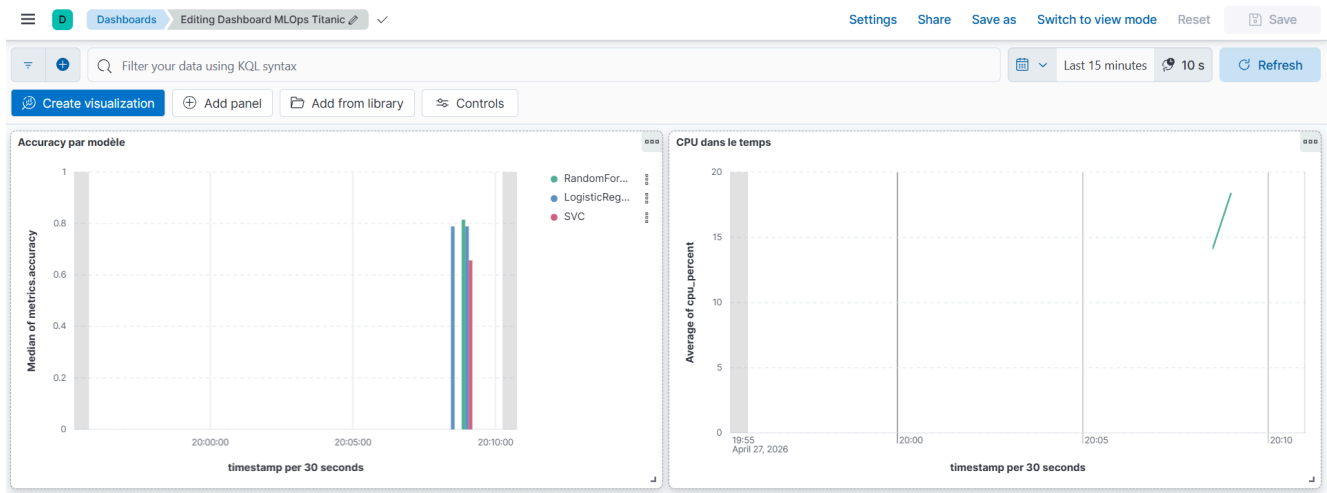
Visualisation 2 — CPU dans le temps (Line Chart)

Ce graphique suit l'utilisation CPU de la machine en temps réel lors de l'entraînement des modèles.

Index	system-metrics
Type	Line chart
Axe X	timestamp (date histogram par 30s)
Axe Y	Average of cpu_percent
Résultat	Pic à ~20% pendant l'entraînement, retour à ~17% après

Captures du Dashboard MLOps Titanic

Vue du dashboard en mode édition — les deux visualisations côte à côte :



Livrables

- ✓ docker-compose.yml — Elasticsearch + Kibana
- ✓ model_pipeline.py — connexion ES + envoi logs MLflow
- ✓ MLflow UI accessible sur http://localhost:5000
- ✓ Data Views : mlflow-metrics + system-metrics dans Kibana
- ✓ Dashboard MLOps Titanic — 2 visualisations
- ✓ Makefile — cibles monitoring-up, monitoring-down, mlflow-server
- ✓ Excellence — monitoring système (CPU, RAM, disque) via psutil

Bonus — Interface React & Pipeline CI/CD Automatique

En complément des ateliers, une interface web React a été développée pour permettre l'ajout de données au dataset et déclencher automatiquement le pipeline de réentraînement.

Architecture

Composant	Port	Rôle
React (frontend)	localhost:3000	Interface utilisateur — formulaire, upload, résultats, graphiques
FastAPI (backend)	localhost:8000	API REST — endpoints /add-data, /upload-file, /run-compare
MLflow	localhost:5000	Suivi des runs de réentraînement
Elasticsearch	localhost:9200	Stockage des logs et métriques
Kibana	localhost:5601	Visualisation du dashboard

Flux CI/CD automatique

```
Utilisateur ajoute données (formulaire / CSV / Excel)
↓
POST /add-data ou POST /upload-file (FastAPI)
↓
Fusion avec titanic.csv → prepare_data()
↓
train_model() → MLflow log params + metrics + model
↓
Elasticsearch reçoit les logs → Kibana mis à jour
↓
React affiche : accuracy + classification report + graphique
```

3 méthodes d'ajout de données

Méthode 1 — Saisie manuelle (plusieurs passagers)

Le formulaire permet de saisir tous les champs du dataset Titanic pour un ou plusieurs passagers simultanément. Un aperçu CSV est affiché en temps réel sous chaque ligne.

Champs obligatoires	pclass, survived, sex, fare, embarked
Champs optionnels	name, age, sibsp, parch, ticket, cabin
Fonctionnalité	Ajouter plusieurs passagers avec + , supprimer avec 
Aperçu temps réel	Chaque ligne affiche un preview CSV avant soumission
Endpoint appelé	POST /add-data (1 passager) ou POST /add-multiple-rows (plusieurs)

 Saisie manuelle

 Fichier CSV

 Fichier Excel

1 passager à ajouter

* Champs obligatoires. Les champs vides seront ignorés.

[+ Ajouter un passager](#)

1 Passager #1

 Exemple CSV : 3,0,"Willey, Mr. Edward",male,,0,0,S.O./P.P. 751,7.55,,S

 Classe *

3 — Troisième classe

 Survécu *

✗ Non (0)

 Sexe *

👤 Male

 Nom complet

Ex: "Willey, Mr. Edward"

 Âge

Ex: 29 (vide si inconnu)

 SibSp (frères/conjoints)

0

 Parch (parents/enfants)

0

 Numéro de billet

Ex: "S.O./P.P. 751"

 Tarif (fare) *

Ex: 7.55

 Cabine

Ex: C85 (vide si inconnue)

 Port d'embarquement *

S — Southampton

 Ajouter 1 passager & Lancer le Pipeline

Méthode 2 — Upload fichier CSV

L'utilisateur peut glisser-déposer un fichier CSV ou le sélectionner via l'explorateur. Le fichier est automatiquement fusionné avec le dataset existant.

Format accepté	.csv
Drag & Drop	Zone de dépôt avec indicateur visuel
Validation	Vérification des colonnes requises avant fusion
Endpoint appelé	POST /upload-file (multipart/form-data)
Exemple ligne	3,0,"Willey, Mr. Edward",male,,0,0,S.O./P.P. 751,7.55,,S

Saisie manuelle

Fichier CSV

Fichier Excel

Importer un fichier CSV

Format attendu (.csv) :

Obligatoires : pclass, survived, sex, fare, embarked

Optionnels : name, age, sibsp, parch, ticket, cabin

titanic.csv

93.4 KB — Prêt à importer

Supprimer le fichier

Importer & Lancer le Pipeline

Méthode 3 — Upload fichier Excel (.xlsx)

Import de fichiers Excel avec fusion automatique. Un fichier de test de 15 passagers a été généré avec mise en forme (survivants en vert, non survivants en rouge) et un onglet Instructions.

Formats acceptés	.xlsx, .xls
Librairie	openpyxl (lecture) — pandas (fusion)
Fichier de test	titanic_test_data.xlsx — 15 passagers, 2 onglets
Endpoint appelé	POST /upload-file (même endpoint que CSV)

Saisie manuelle

Fichier CSV

Fichier Excel

Importer un fichier Excel

Format attendu (.xlsx / .xls) :

Obligatoires : pclass, survived, sex, fare, embarked

Optionnels : name, age, sibsp, parch, ticket, cabin



✓ titanic_test_data.xlsx

7.2 KB — Prêt à importer

 Supprimer le fichier

Importer & Lancer le Pipeline

Affichage des résultats

Après chaque déclenchement du pipeline, les résultats sont affichés en temps réel dans le panneau de droite :

- Accuracy du modèle en grand (ex: 79.89%)
- Nombre de lignes ajoutées et total du dataset
- Classification Report complet (Precision, Recall, F1-Score, Support)
- Weighted Average mis en évidence

Résultats du Pipeline

✓ Fichier 'titanic_test_data.xlsx' importé avec succès !

80.00%

Accuracy du modèle

+15 lignes ajoutées

Total : 1324 lignes

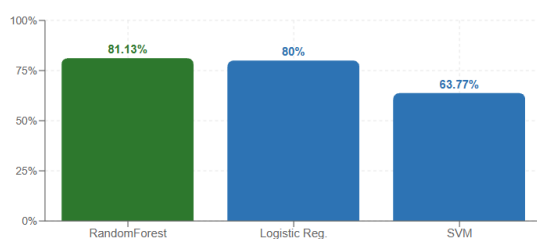
Classification Report

Classe	Precision	Recall	F1-Score	Support
✗ Non survécu	0.796	0.875	0.834	152
✓ Survécu	0.806	0.699	0.749	113
Weighted Avg	0.801	0.800	0.798	265

Comparaison des modèles

Le bouton '🔄 Comparer les modèles' lance une comparaison des 3 algorithmes et affiche un graphique en barres avec le classement final.

Comparaison des modèles



🔄 Comparer les modèles

Rang	Modèle	Accuracy	F1 (w)	Recall (w)
1	RandomForest	81.1%	80.7%	81.1%
2	Logistic Reg.	80.0%	79.8%	80.0%
3	SVM	63.8%	59.3%	63.8%

🏆 Meilleur modèle : RandomForest — 81.13%

Stack technique

Frontend	React 18 + recharts + axios
Backend	FastAPI + python-multipart + openpyxl
Endpoints	/add-data, /add-multiple-rows, /upload-file, /run-compare, /dataset-info
CI/CD	Ajout données → pipeline auto → MLflow + Elasticsearch → Kibana
Lancement	uvicorn app:app --reload --port 8000 + npm start (port 3000)

Livrables

- ✓ app.py : 7 endpoints (health, predict, retrain, add-data, add-multiple-rows, upload-file, run-compare)
- ✓ frontend/src/App.js — Interface React avec 3 méthodes d'ajout
- ✓ titanic_test_data.xlsx — Fichier Excel de 15 passagers pour test
- ✓ Pipeline CI/CD automatique déclenché à chaque ajout de données

Récapitulatif — Bilan des Livrables

Structure du projet

```
MLOps_Project/
├── data/titanic.csv
├── models/titanic_model.pkl
├── src/model_pipeline.py    ← Atelier 2 + 5 + 7
├── tests/test_model.py     ← Excellence Atelier 2
├── conftest.py             ← Excellence Atelier 2
├── app.py                  ← Atelier 4
├── main.py                 ← Atelier 2 + 3
├── Makefile                ← Atelier 3 + 4 + 5 + 6 + 7
├── Dockerfile              ← Atelier 6
├── docker-compose.yml      ← Atelier 7
├── .env                    ← Excellence Atelier 6
├── .dockerignore
├── requirements.txt
├── logs/pipeline.log       ← Excellence Atelier 2
└── mlflow.db               ← Excellence Atelier 5
```

Points d'Excellence — Description et Intérêt

Excellence Atelier 2 — Tests Unitaires & Logging

Ce qui a été fait	13 tests unitaires couvrant <code>prepare_data()</code> , <code>train_model()</code> , <code>evaluate_model()</code> , <code>save_model()</code> et <code>load_model()</code> . Remplacement de tous les <code>print()</code> par un logger Python (<code>logging.basicConfig</code>) qui écrit dans le terminal ET dans <code>logs/pipeline.log</code> .
Intérêt	Les tests garantissent que chaque modification du code ne casse pas les fonctions existantes (régression). Le logging permet de tracer l'historique d'exécution, de déboguer en production et de conserver une trace persistante des traitements sans polluer la sortie standard.
Résultat	✓ 13/13 tests passés — <code>pytest tests/ -v</code>

Excellence Atelier 3 — Makefile CI Complet

Ce qui a été fait	Cible 'CI' qui enchaîne automatiquement : <code>black</code> (formatage) → <code>flake8</code> (qualité) → <code>bandit</code> (sécurité) → <code>pytest --cov</code> (tests + couverture). Cible 'clean' qui supprime <code>__pycache__</code> , <code>.pyc</code> , <code>mlruns/</code> , <code>htmlcov/</code> . Rapport HTML de couverture de code généré dans <code>htmlcov/</code> .
Intérêt	Une seule commande 'make ci' exécute toute la chaîne de validation du code. Cela reproduit localement ce qu'un pipeline CI/CD (Jenkins, GitHub Actions) ferait automatiquement, garantissant un code propre, sécurisé et testé avant chaque déploiement.
Résultat	✓ make ci — pipeline complet en une commande

Excellence Atelier 4 — FastAPI /health + Validation Pydantic

Ce qui a été fait	Endpoint GET /health qui vérifie que l'API est en ligne et que le modèle est bien chargé. Validation Pydantic stricte sur /predict : Pclass (1-3), Age (0-120), Fare (>=0), SibSp/Parch (0-10), solver valide pour /retrain. Ajout du champ 'probability' dans la réponse /predict. Endpoint /retrain exposé avec validation des hyperparamètres.
Intérêt	/health permet aux orchestrateurs (Kubernetes, Docker) de vérifier automatiquement si le service est opérationnel. La validation Pydantic rejette les données invalides avant qu'elles atteignent le modèle, évitant des erreurs silencieuses et renforçant la robustesse de l'API en production.
Résultat	✓ <pre>{"status": "ok", "model_loaded": true, "api_version": "1.0.0" }</pre>

Excellence Atelier 5 — Comparaison de Modèles MLflow

Ce qui a été fait	Fonction compare_models() qui entraîne et compare 3 algorithmes : LogisticRegression, RandomForestClassifier et SVM. Chaque run est loggué séparément dans MLflow avec ses propres hyperparamètres et métriques. Le meilleur modèle est automatiquement sauvegardé. Flag --compare ajouté dans main.py.
Intérêt	La comparaison de modèles est au cœur du processus MLOps : elle permet de choisir objectivement le meilleur algorithme pour un problème donné. MLflow visualise les runs côte à côte dans son interface, facilitant l'analyse des performances et la prise de décision pour le déploiement.
Résultat	✓ python main.py --compare — 3 runs MLflow, meilleur modèle auto-sauvegardé

Checklist finale

Livrable	Statut	Atelier
src/model_pipeline.py modularisé	✓	Atelier 2
main.py avec args CLI (--prepare/--train/--evaluate/--all)	✓	Atelier 2
Makefile : install, train, evaluate, format, lint, security	✓	Atelier 3
app.py — endpoint /predict	✓	Atelier 4
Commande make run-api	✓	Atelier 4
mlflow.log_param() + log_metric() + log_model()	✓	Atelier 5
Commande make mlflow-ui (backend SQLite)	✓	Atelier 5
Dockerfile — python:3.11-slim + uvicorn	✓	Atelier 6
Image Docker publiée sur Docker Hub	✓	Atelier 6
docker-compose.yml — Elasticsearch + Kibana	✓	Atelier 7
model_pipeline.py — log_to_elasticsearch() + log_system_metrics()	✓	Atelier 7

Makefile — monitoring-up, monitoring-down, mlflow-server	✓	Atelier 7
★ Tests unitaires 13/13 + Logging (logs/pipeline.log)	✓	Excellence A2
★ Makefile CI : make ci (format+lint+security+coverage)	✓	Excellence A3
★ FastAPI /health + Validation Pydantic + /retrain	✓	Excellence A4
★ Comparaison RandomForest / LR / SVM via MLflow	✓	Excellence A5